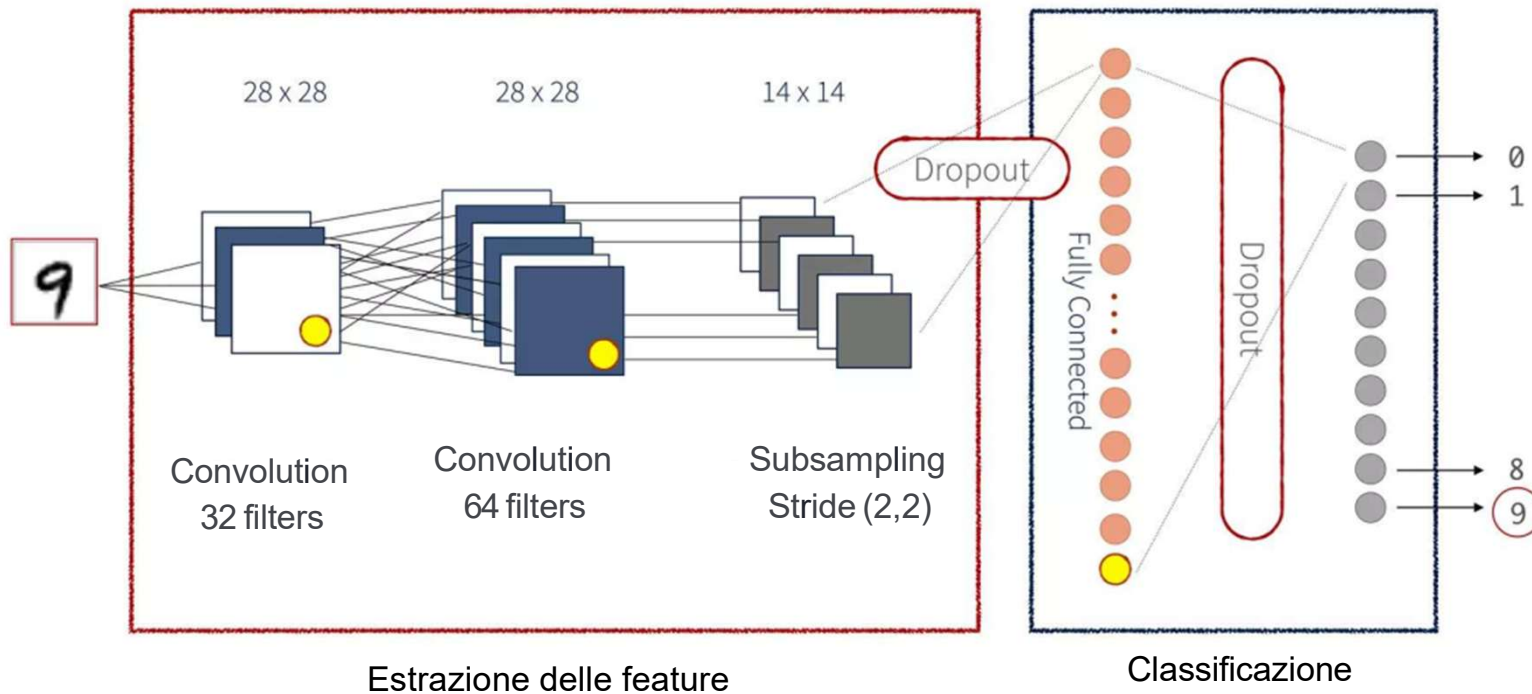


Introduzione alle Reti Neurali

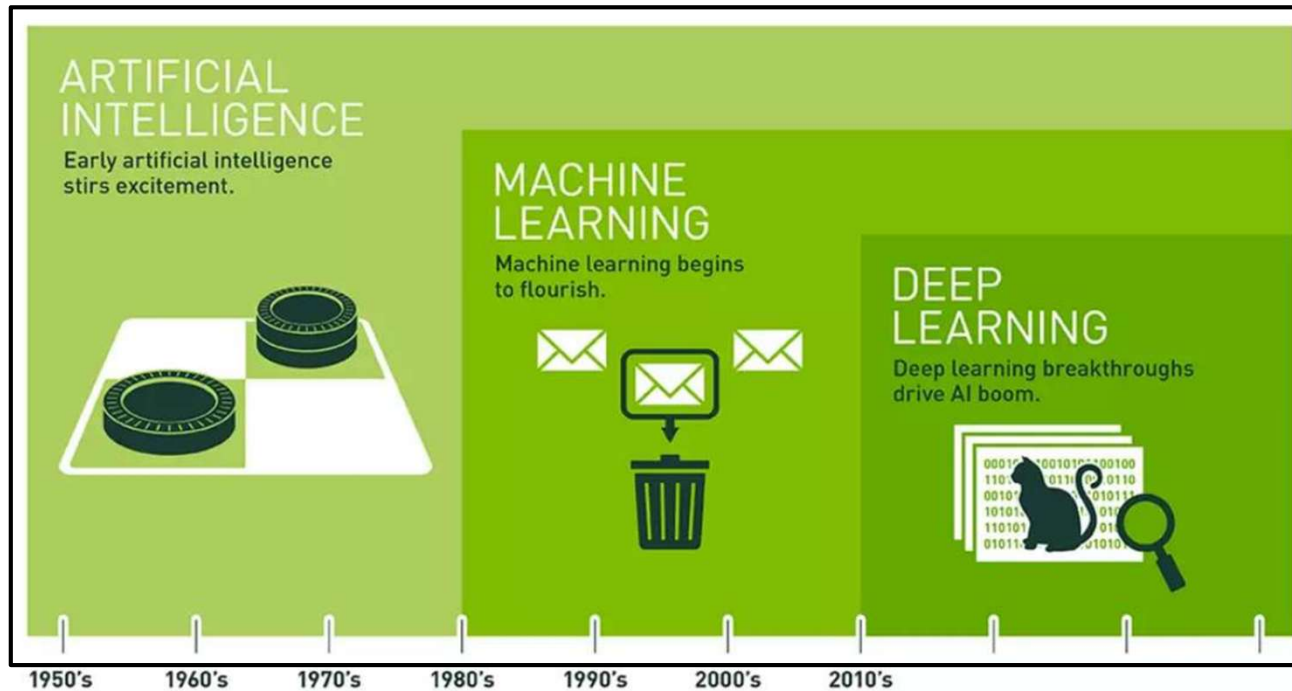
Introduzione alle Reti Neurali

- Cos'è il Deep Learning?
- Cosa può fare il Deep Learning per te?
- Cose sono le reti neurali artificiali?
- Iniziamo con il percettrone
- Capire l'effetto delle funzioni di attivazione

Esempio di rete neurale convoluzionale

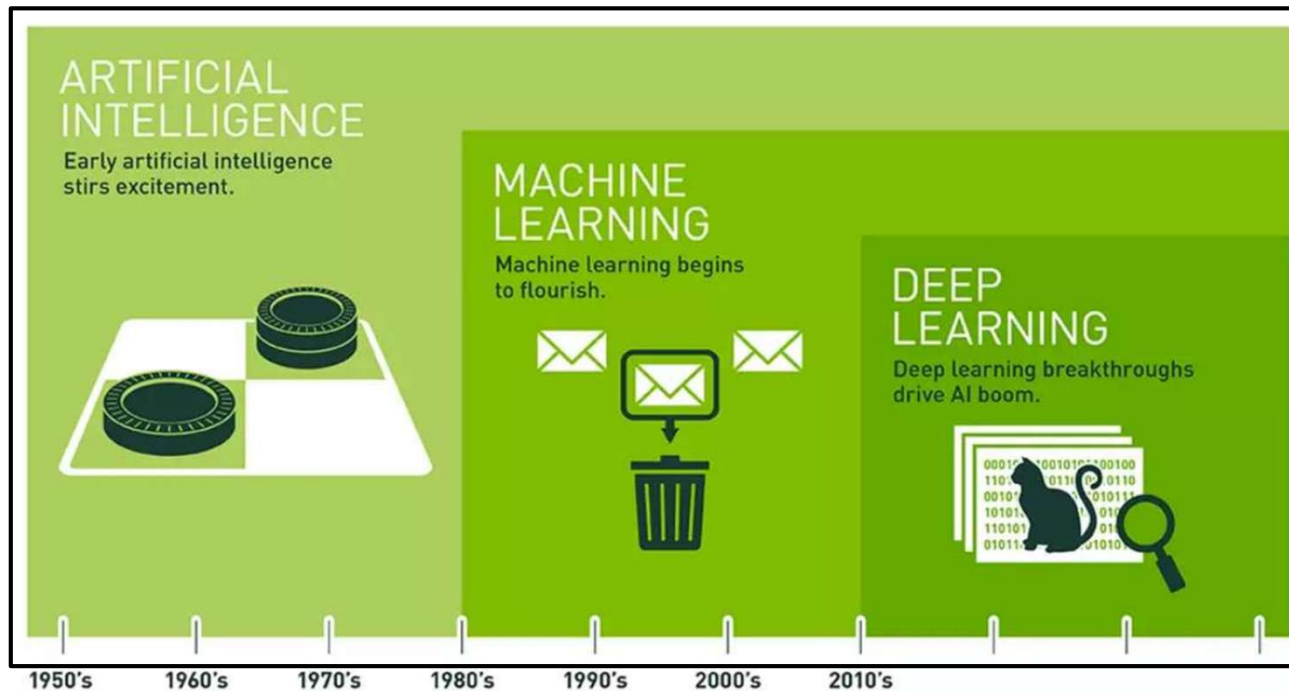


Deep Learning, ML, AI, ... aiuto!



Da un'iniziale scarica di ottimismo negli anni '50, vari sottoinsiemi dell'intelligenza artificiale – prima il machine learning, poi il deep learning, un sottoinsieme del machine learning – anno creato sconvolgimenti sempre più grandi.

Deep Learning, ML, AI, ... aiuto!



Da un'iniziale scarica di ottimismo negli anni '50, vari sottoinsiemi dell'intelligenza artificiale – prima il machine learning, poi il deep learning, un sottoinsieme del machine learning – anno creato sconvolgimenti sempre più grandi.

Intelligenza Artificiale

L'intelligenza artificiale è
l'intelligenza umana esibita dalle
macchine

Thinking Humanly

“The exciting new effort to make computers think ... *machines with minds*, in the full and literal sense.” (Haugeland, 1985)

“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ...” (Bellman, 1978)

Acting Humanly

“The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990)

“The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)

Thinking Rationally

“The study of mental faculties through the use of computational models.” (Charniak and McDermott, 1985)

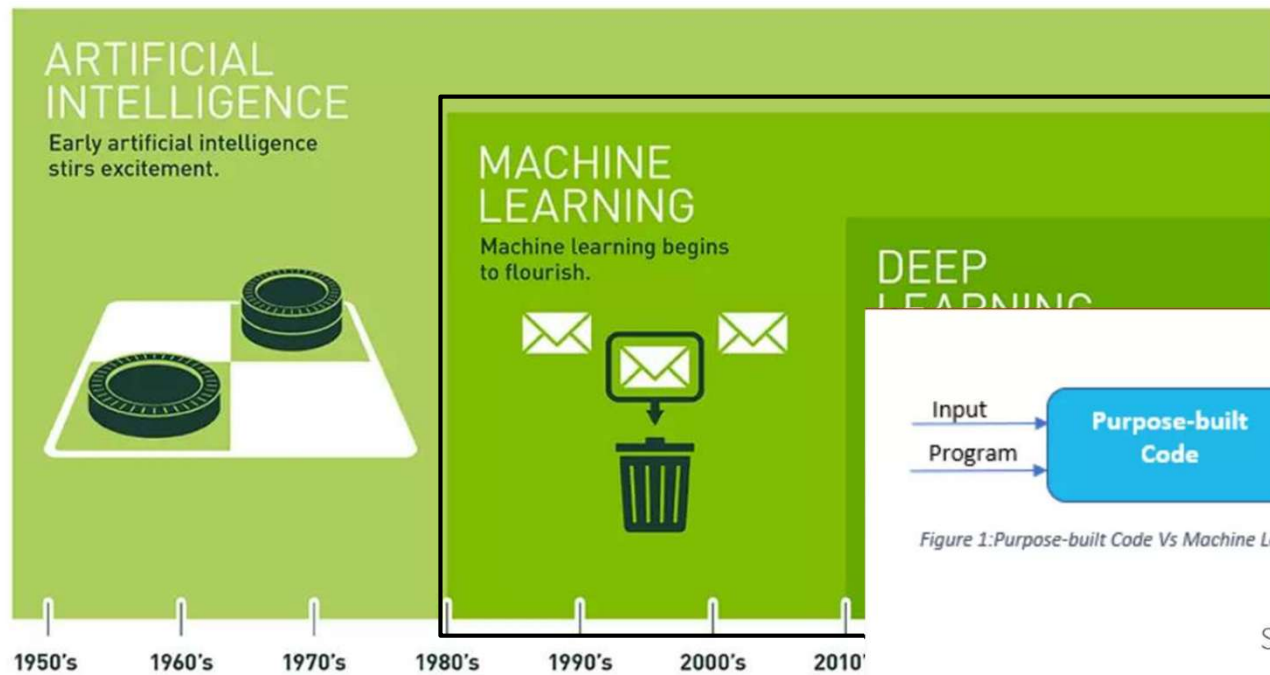
“The study of the computations that make it possible to perceive, reason, and act.” (Winston, 1992)

Acting Rationally

“Computational Intelligence is the study of the design of intelligent agents.” (Poole *et al.*, 1998)

“AI ... is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

Deep Learning, ML, AI, ... aiuto!



Da un'iniziale scarica di ottimismo negli anni '50, vari sottoinsiemi dell'intelligenza artificiale – prima il machine learning, poi il deep learning, un sottoinsieme del machine learning – anno creato sconvolgimenti sempre più grandi.

Machine Learning

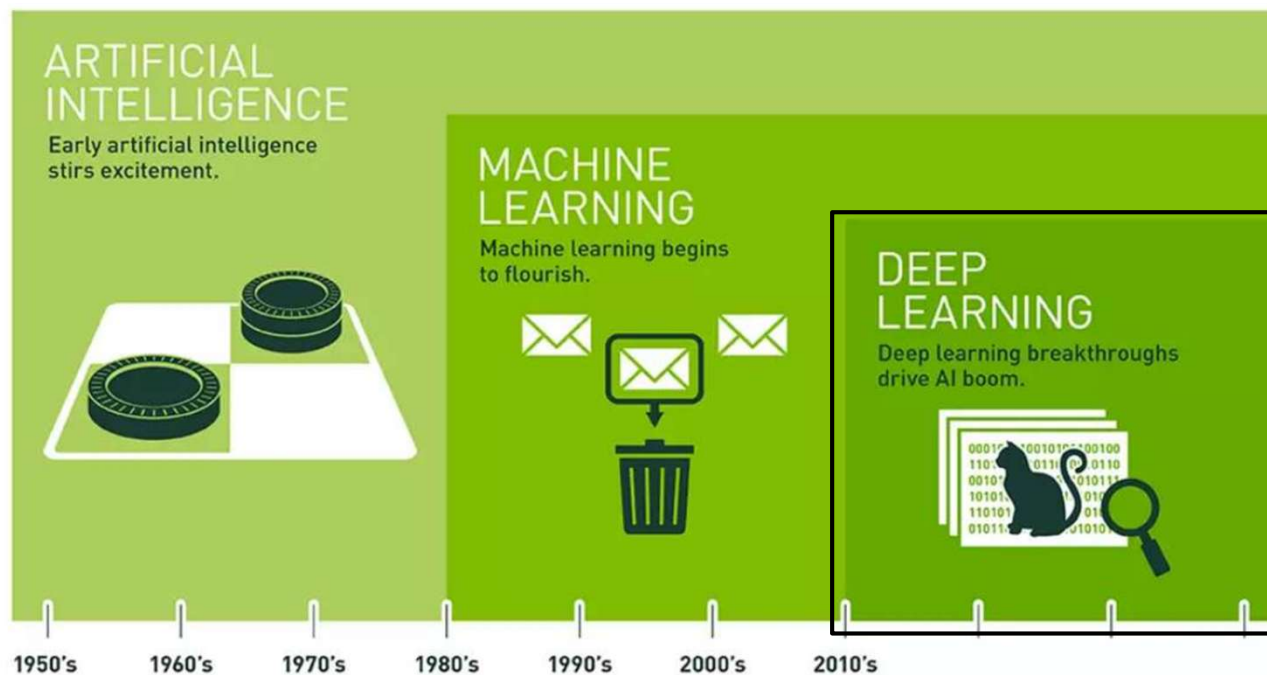
La branca di studi per dare ai computer l'abilità di apprendere senza essere esplicitamente programmati.



Figure 1: Purpose-built Code Vs Machine Learning Model

Source: <https://bit.ly/2NnyrOz>

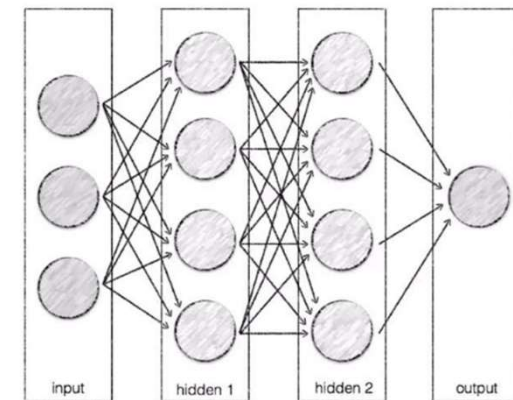
Deep Learning, ML, AI, ... aiuto!



Da un'iniziale scarica di ottimismo negli anni '50, vari sottoinsiemi dell'intelligenza artificiale – prima il machine learning, poi il deep learning, un sottoinsieme del machine learning – hanno creato sconvolgimenti sempre più grandi.

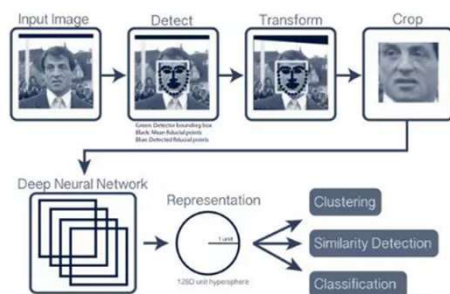
Deep Learning

Usare le reti neurali deep per implementare il machine learning

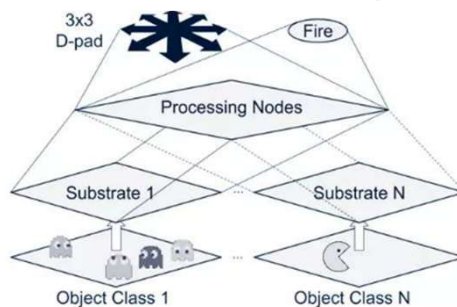


Applicazioni delle reti neurali

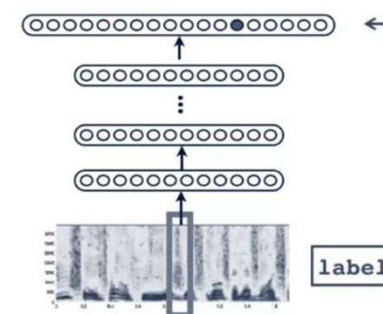
Riconoscimento di oggetti: volti



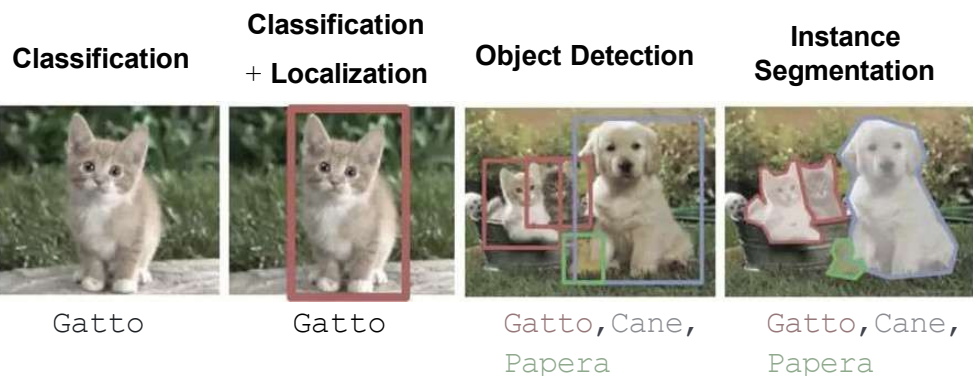
Giocare a videogiochi



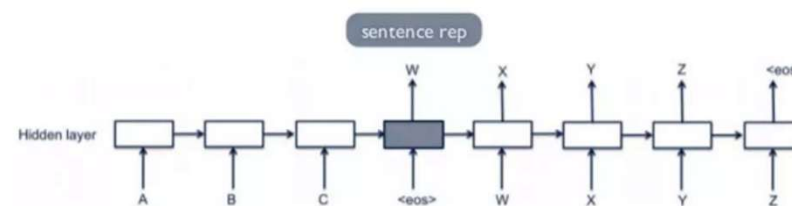
Riconoscimento di oggetti: parlato



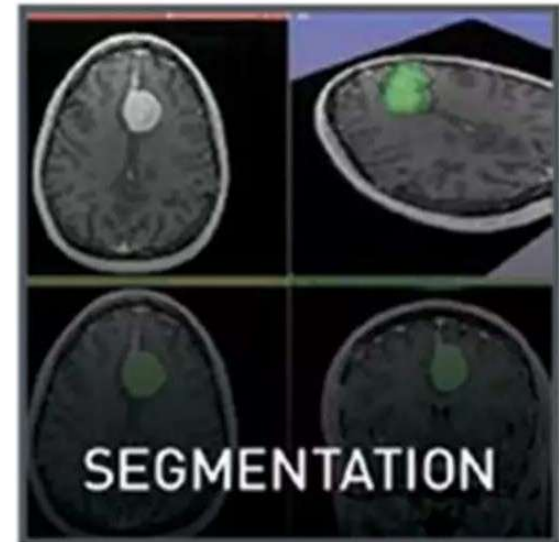
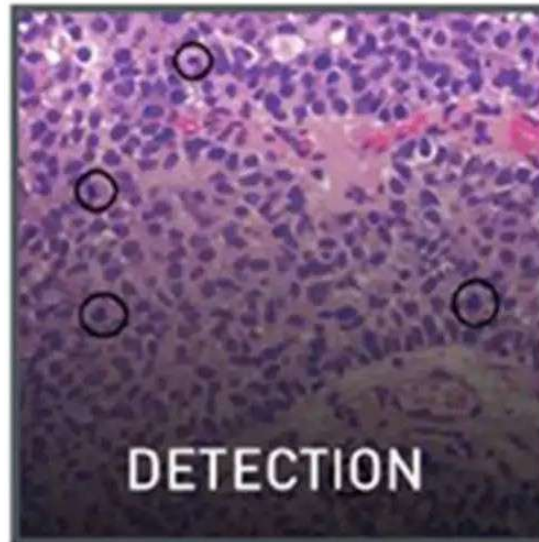
Classificazione di oggetti (object classification)



Traduzione del linguaggio



Sanità



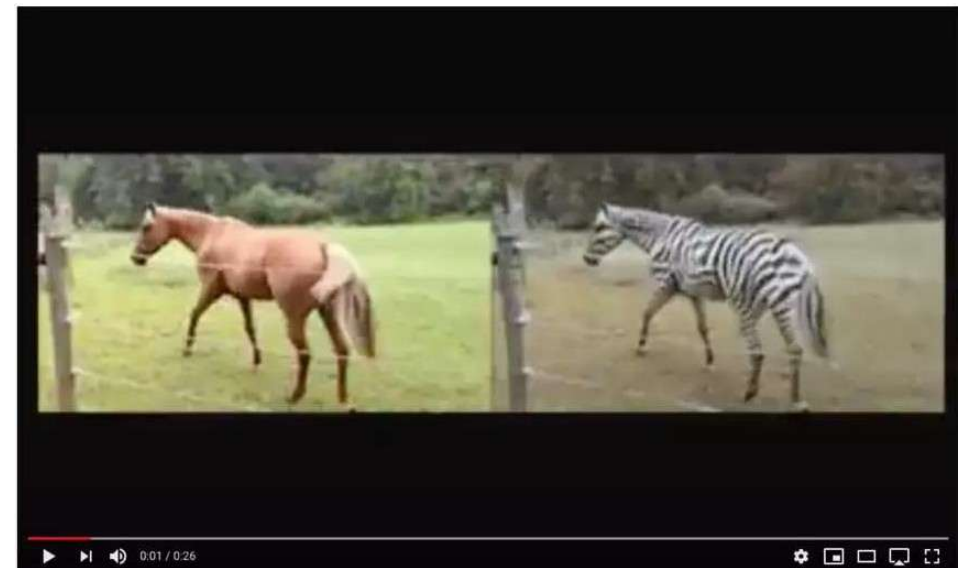
Falsificare video

Video falso verosimile di Barack Obama generato dall'AI



Fonte: <https://youtu.be/dkoi7sZvWiU>

Trasformare il video di un cavallo nel video di una zebra in tempo reale tramite le GAN



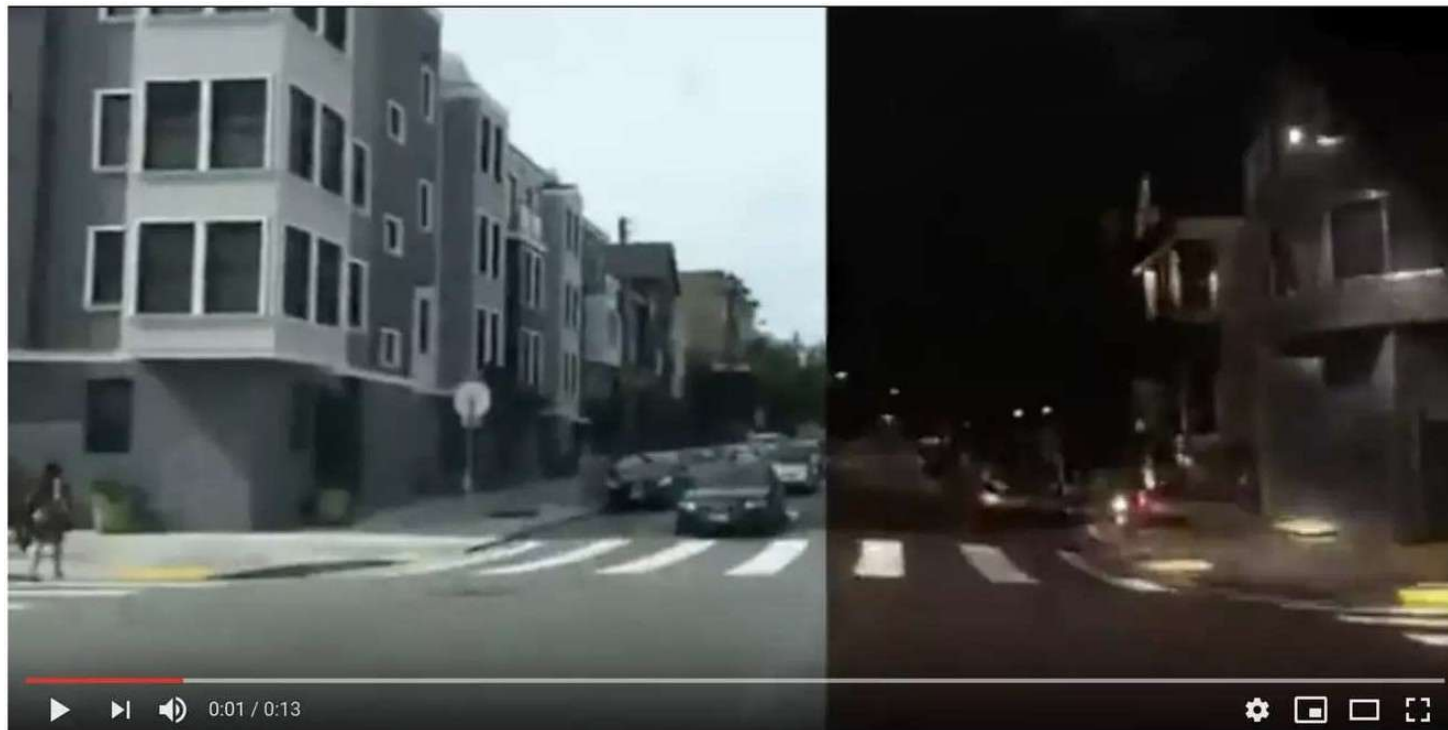
Fonte:
<https://www.youtube.com/watch?v=9reHvktowLY>

Auto con guida automatica



Source: <https://youtu.be/URmxzxYlmtg?t=6m30s>

Trasformare il giorno in notte



Source: <https://youtu.be/N7KbfWodXJE>

Concettualmente, cosa sono le reti neurali artificiali?

- Un neurone riceve un segnale, lo elabora, e poi propaga il segnale (oppure no)
- Il cervello è composto di circa 100 miliardi di neuroni, ognuno connesso a ~10K altri neuroni: 10^{15} connessioni sinaptiche
- Le reti neurali artificiali (ANN) sono una imitazione semplicistica di un cervello composto da una rete densa di strutture semplici



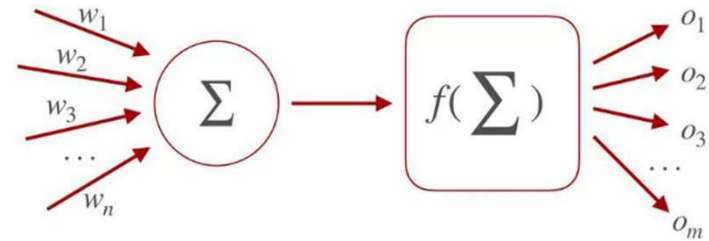
Source: <http://bit.ly/2ijGlnD>

Modello matematico concettuale

- Riceve un input da n fonti
- Calcola la somma pesata

$$h_1 = x_1 w_1 + x_2 w_2 + \dots x_n w_n$$

- Passa attraverso una *funzione di attivazione*
- Invia il segnale a m successivi neuroni



"Paralleli"

Un singolo neurone nel cervello è una macchina incredibilmente complessa che ancora oggi non comprendiamo. Un singolo “neurone” in una rete neurale è una funzione matematica estremamente semplice che cattura una minuscola frazione della complessità di un neurone biologico. Quindi dire che le reti neurali imitano il cervello è vero a livello di vaga ispirazione, ma le reti neurali artificiali non replicano minimamente il vero cervello biologico.

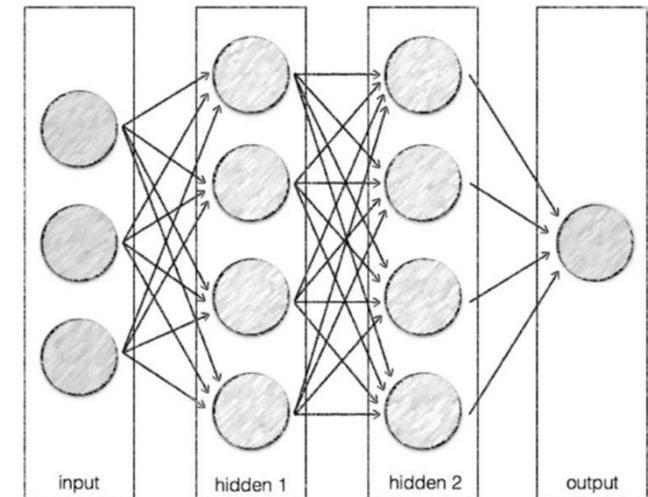
A single neuron in the brain is an incredibly complex machine that even today we don't understand. A single "neuron" in a neural network is an incredibly simple mathematical function that captures a minuscule fraction of the complexity of a biological neuron. So to say neural networks mimic the brain, that is true at the level of loose inspiration, but really artificial neural networks are nothing like what the biological brain does.

Andrew Ng

Medium Article: [Google Brain's Co-inventor Tells Why He's Building Chinese Neural Networks](#)

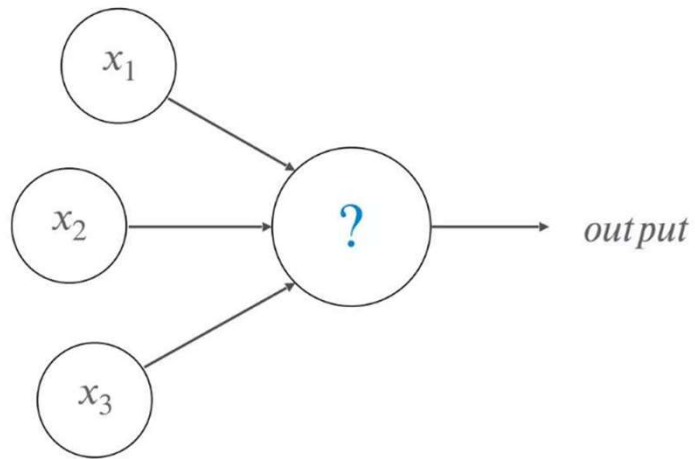
Rete Neurale Artificiale

- Organizzata in strati (*layer*) di neuroni come un modello black-box (a scatola nera)
- Tipicamente 3 o più: input, hidden (intermedio) e output
- Per il riconoscimento di immagini,
 - i primi layer sono organizzati simili alla corteccia visiva,
 - ma le funzioni di attivazione sono più semplici per le ANNs rispetto alle reti neurali biologiche



Percettrone

Neurone artificiale binario (semplificato)



Faccio snowboard questo weekend?

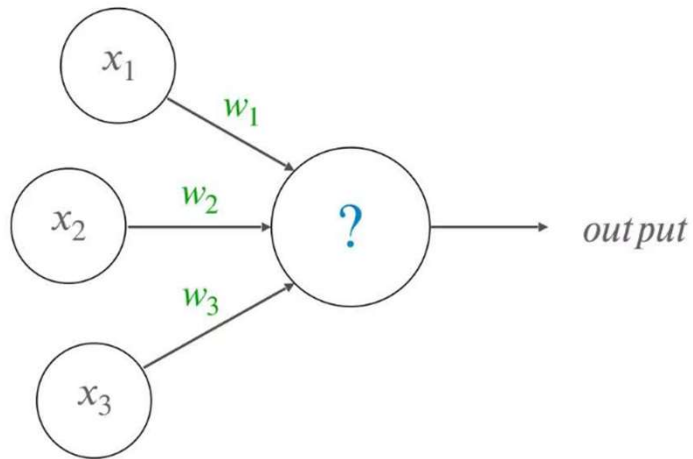
$X_1 \rightarrow$ Il meteo è buono?

$X_2 \rightarrow$ La neve è buona?

$X_3 \rightarrow$ Me la sento di guidare?

Percettrone

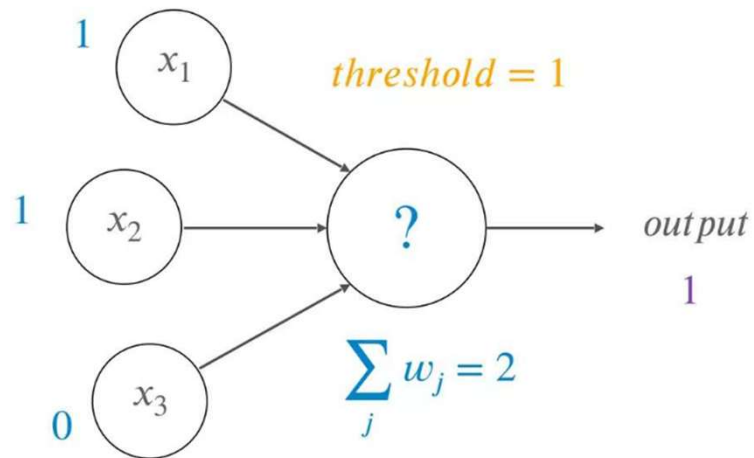
Neurone artificiale binario (semplificato) **con pesi**



$$output = \begin{cases} 0, & \sum_{j=0}^n w_j x_j \leq \textbf{soglia} \\ 1, & \sum_{j=0}^n w_j x_j > \textbf{soglia} \end{cases}$$

Percettrone

Neurone artificiale binario (semplificato)



Faccio snowboard questo weekend?

$X_1 \rightarrow 1$ (buon meteo)

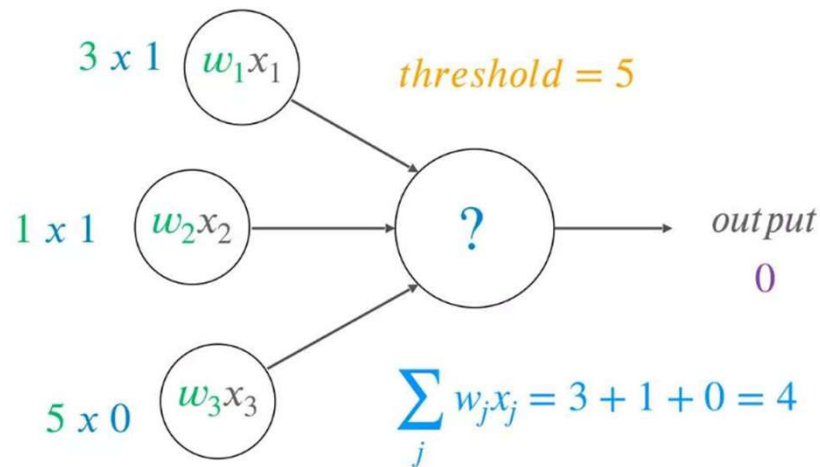
$X_2 \rightarrow 1$ (molta neve)

$X_3 \rightarrow 0$ (strada ghiacciata)

Percettrone

Neurone artificiale binario (semplificato) **con pesi**

Persona: Sciatore occasionale



Faccio snowboard questo weekend?

$x_1 = 1$ (*buon meteo*)

$w_1 = 3$

$x_2 = 1$ (*molta neve*)

$w_2 = 1$

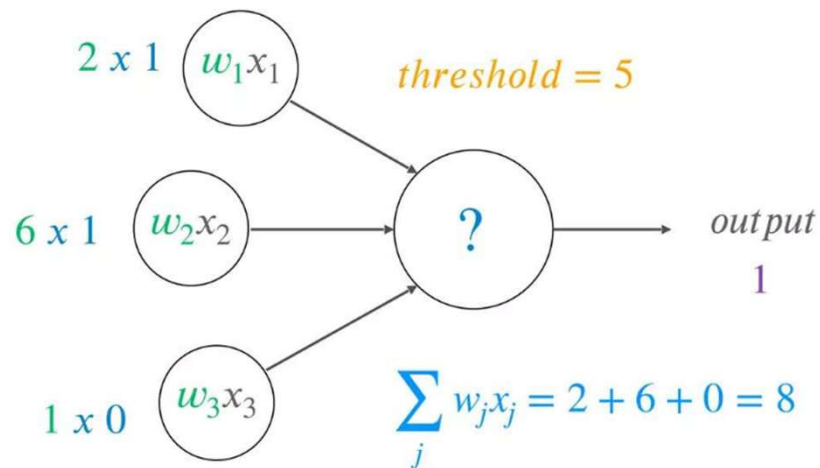
$x_3 = 0$ (*strada ghiacciata*)

$w_3 = 5$

Percettrone

Neurone artificiale binario (semplificato) con pesi

Persona: Sciatore appassionato



Faccio snowboard questo weekend?

$x_1 = 1$ (buon meteo)

$w_1 = 2$

$x_2 = 1$ (molta neve)

$w_2 = 6$

$x_3 = 0$ (strada ghiacciata)

$w_3 = 1$

Introdurre il Bias

I perceptron hanno bisogno di includere il bias

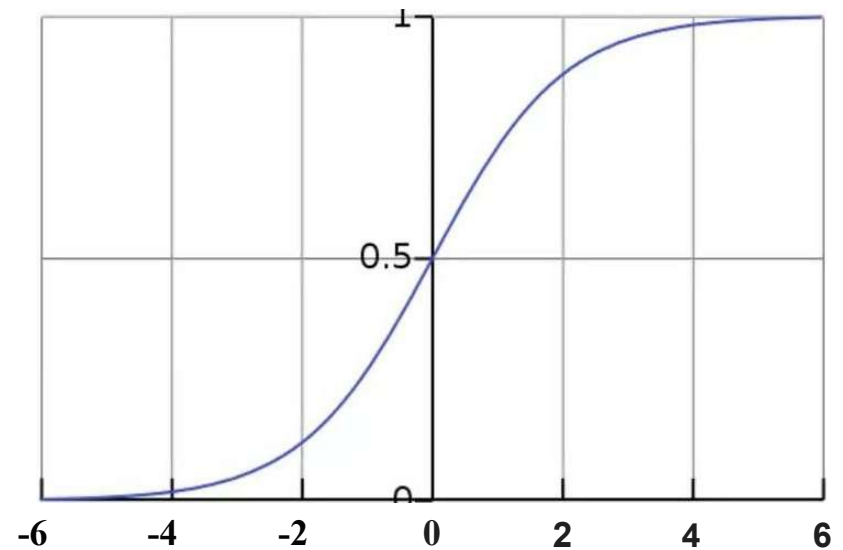
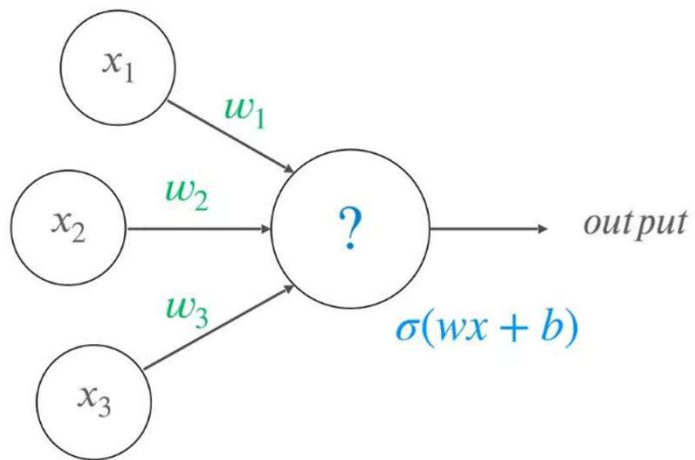
$$output = \begin{cases} 0, & wx + b \leq 0 \\ 1, & wx + b > 0 \end{cases}$$

Dove b rappresenta quanto è facile far sì che il perceptrone si attivi

es. lo sciatore appassionato ha un forte bias per andare a sciare mentre lo sciatore occasionale non ha un bias così forte

Neurone Sigmoidoide

Il neurone artificiale più comune

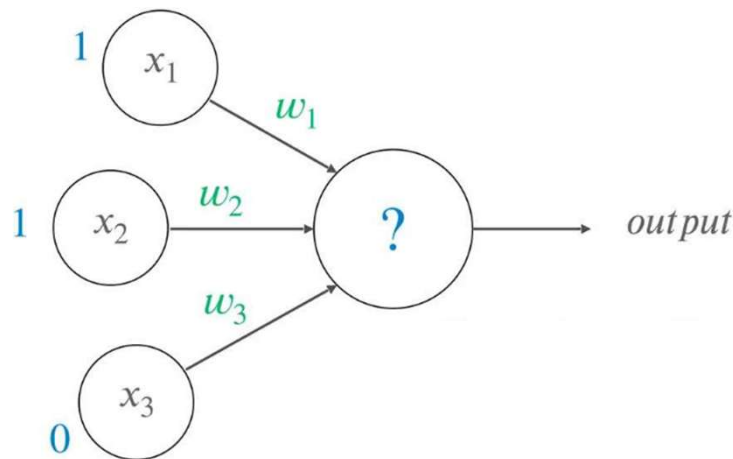


Anzichè $[0, 1]$, ora $(0, \dots, 1)$

Dove l'output è definito da $\sigma(\mathbf{wx} + \mathbf{b})$

Neurone Sigmoido

Persona: Sciatore
appassionato



$$? = \sigma(wx + b) = \sigma(1 \times 0.3 + 1 \times 0.6 + 0 \times 0.1) = \sigma(0.9) = 0.7109$$

Faccio snowboard questo weekend?

$x_1 = 1$ (*buon meteo*)

$w_1 = 2$

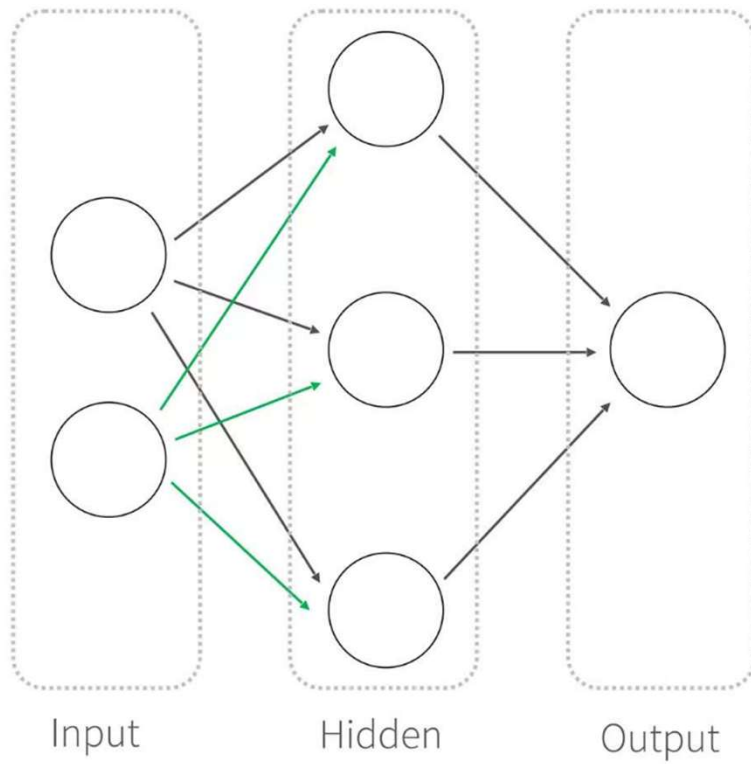
$x_2 = 1$ (*molta neve*)

$w_2 = 6$

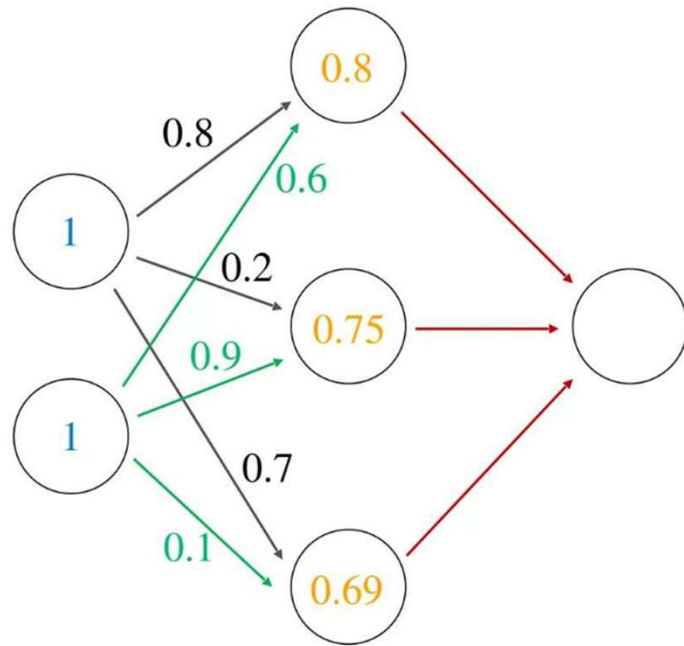
$x_3 = 0$ (*strada ghiacciata*)

$w_3 = 1$

ANN da due layer (semplificata)



ANN da due layer (semplificata)

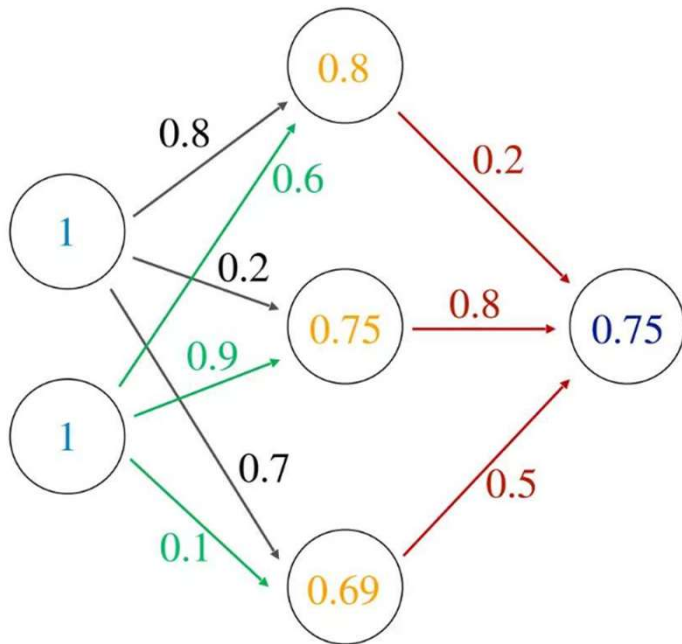


$$h_1 = \sigma(1 \times 0.8 + 1 \times 0.6) = 0.80$$

$$h_2 = \sigma(1 \times 0.2 + 1 \times 0.9) = 0.75$$

$$h_3 = \sigma(1 \times 0.7 + 1 \times 0.1) = 0.69$$

ANN da due layer (semplificata)



$$\begin{aligned} out &= \sigma(0.2 \times 0.8 + 0.8 \times 0.75 + 0.5 \times 0.69) \\ &= \sigma(1.105) \\ &= 0.75 \end{aligned}$$

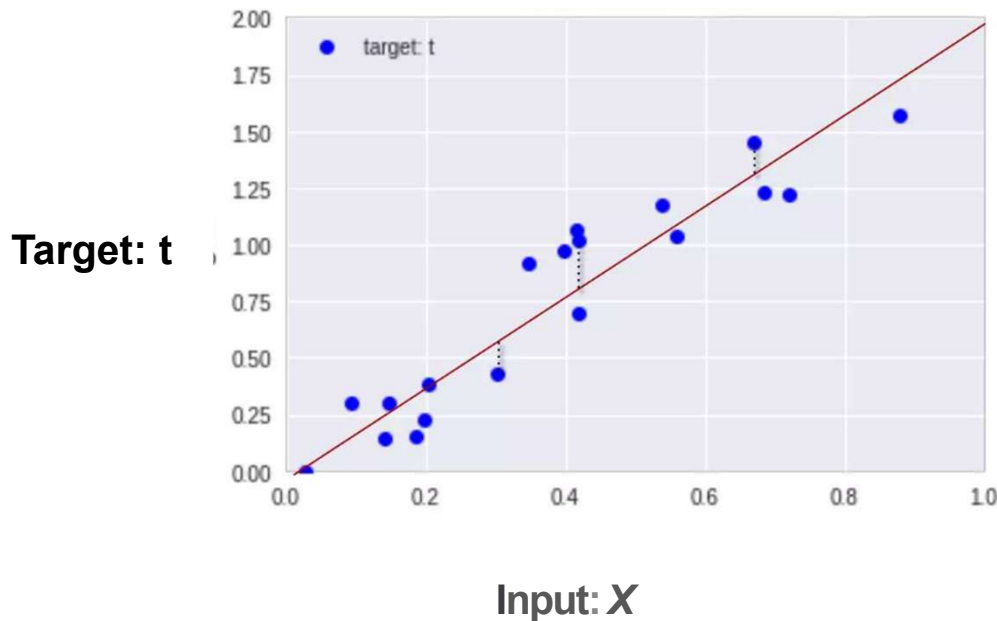
Il potere immenso delle strutture semplici

Ottime risorse

- Andrej Karpathy's [CS231N Neural Networks](#)
- Steve Miller's [How to build a neural network](#)
- Michael Nielsen's [Neural Networks and Deep Learning, Chapter 1](#)

Funzione di costo

Dati etichettati



Per questo esempio di regressione, vogliamo determinare il migliore p (pendenza della linea) per

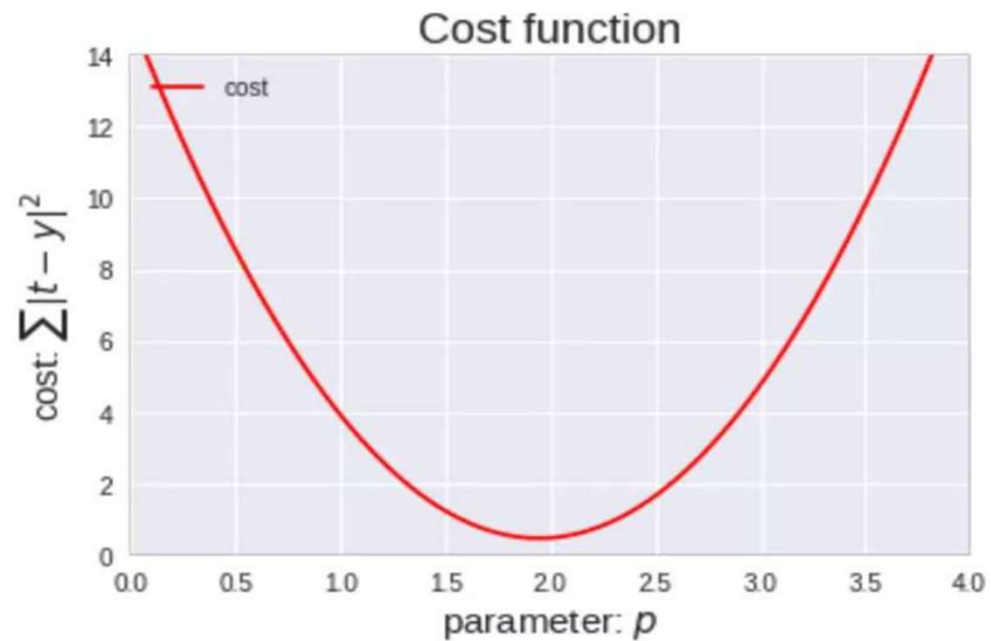
$$y = x \cdot p$$

Vogliamo calcolare la funzione di costo, come l'errore quadratico medio, l'errore assoluto medio, l'errore di bias medio, la SVM loss, ecc

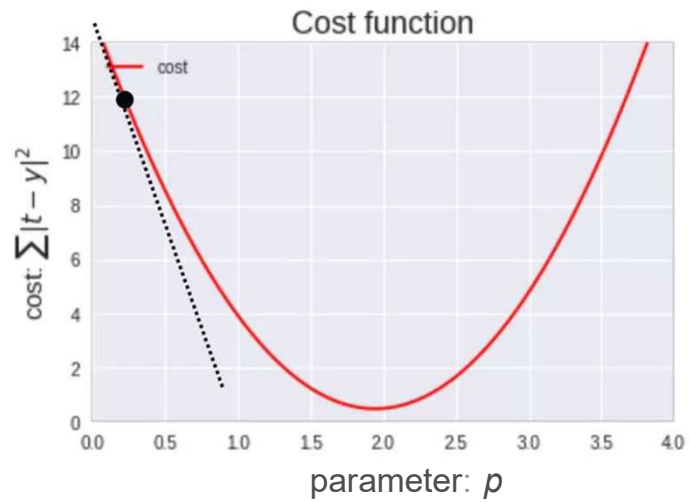
Per questo esempio, usiamo la somma delle differenze assolute quadratiche

$$cost = \sum (t-y)^2$$

Visualizziamo questa funzione di costo

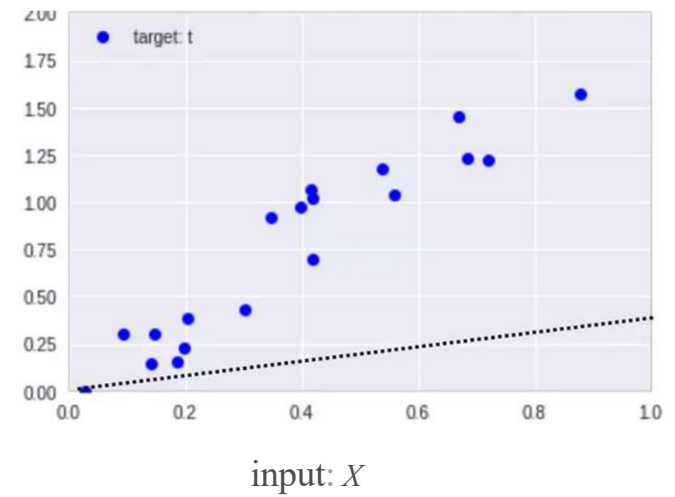


Calcolare la sua derivata

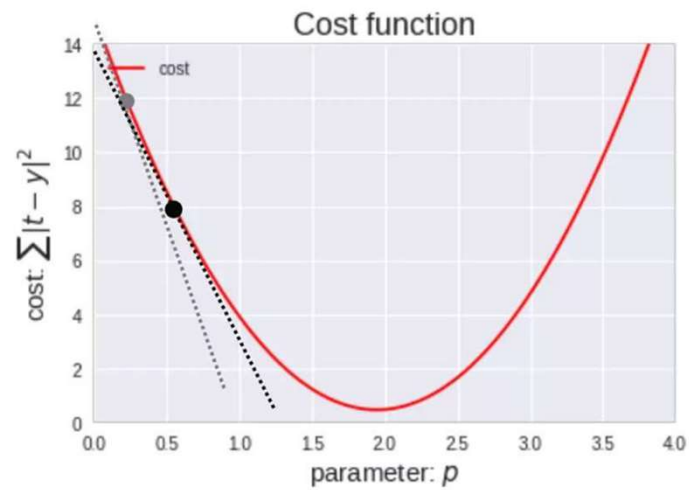


Target: t

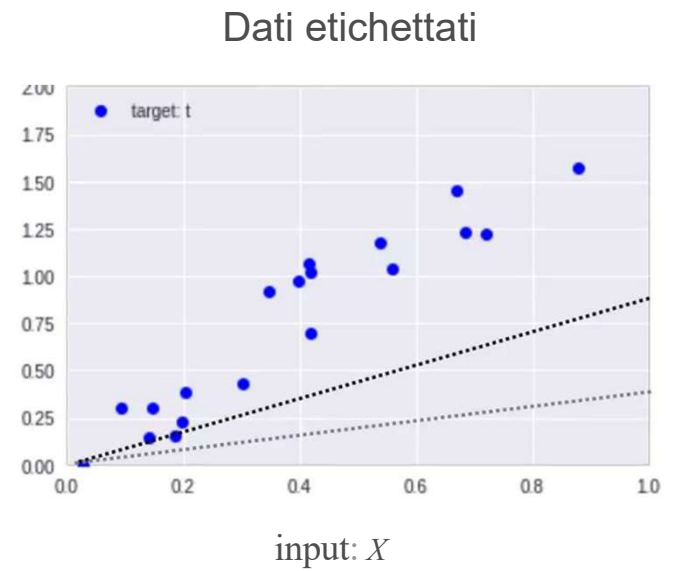
Dati etichettati



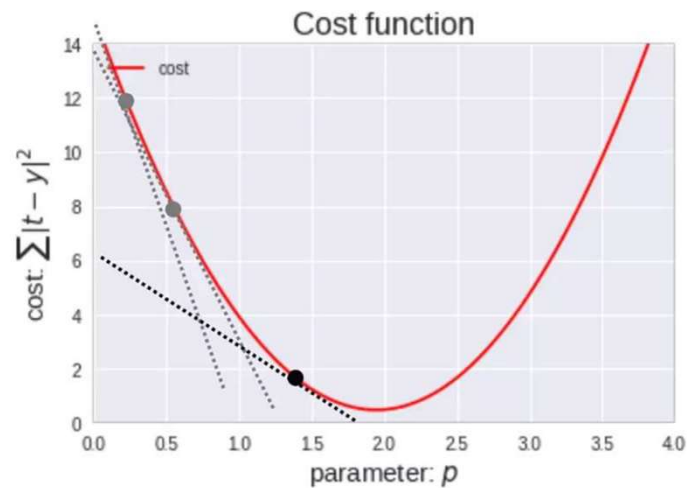
Discesa del gradiente



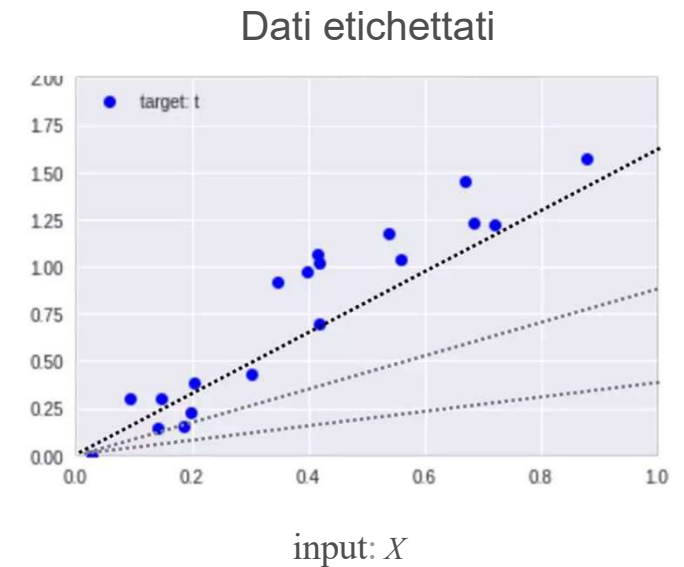
Target: t



Discesa del gradiente



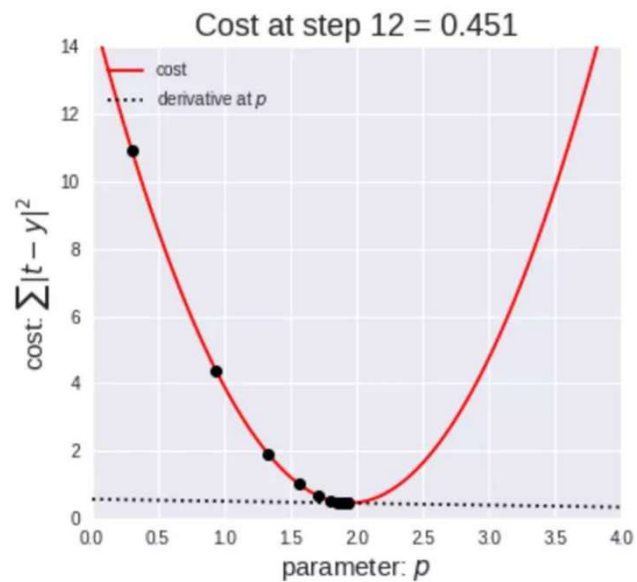
Target: t



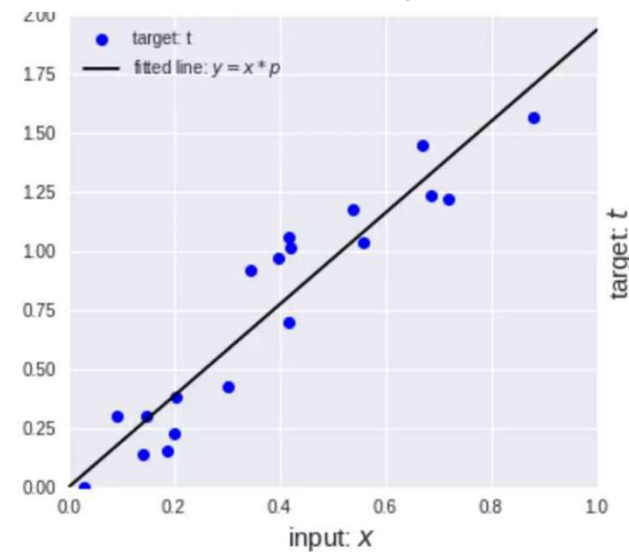
L'obiettivo è trovare il punto più basso della funzione di costo (ovvero il *punto di minimo*).

La Discesa del gradiente iterativamente (un passo alla volta) scende giù per la curva della funzione di costo per trovare il punto di minimo

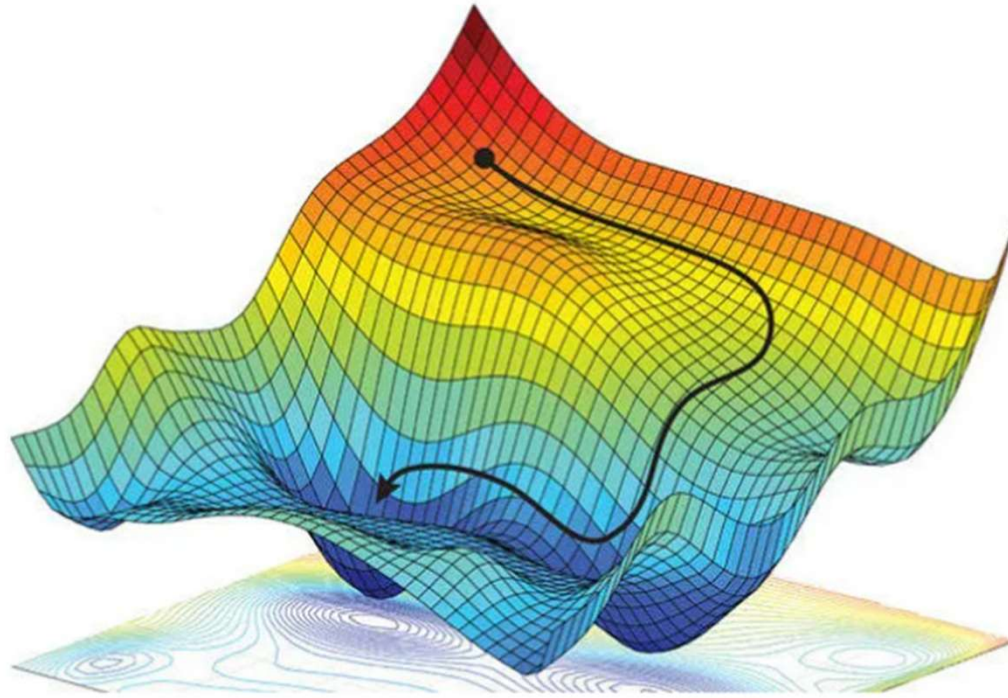
Ottimizzazione della discesa del gradiente



Dati etichettati e output del modello

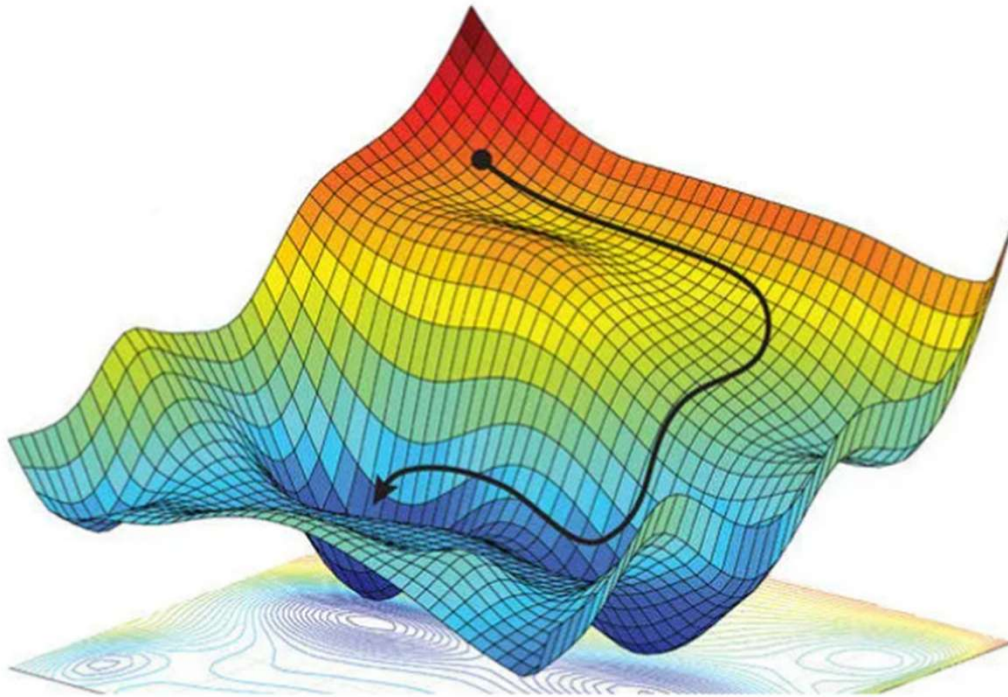


Ottimizzazione della discesa del gradiente



$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b_2}, \dots, \frac{\partial L}{\partial w_m}, \frac{\partial L}{\partial b_n} \right]$$

Ottimizzazione della discesa del gradiente



$$w_{new} = w_{old} + \Delta w$$

Learning rate

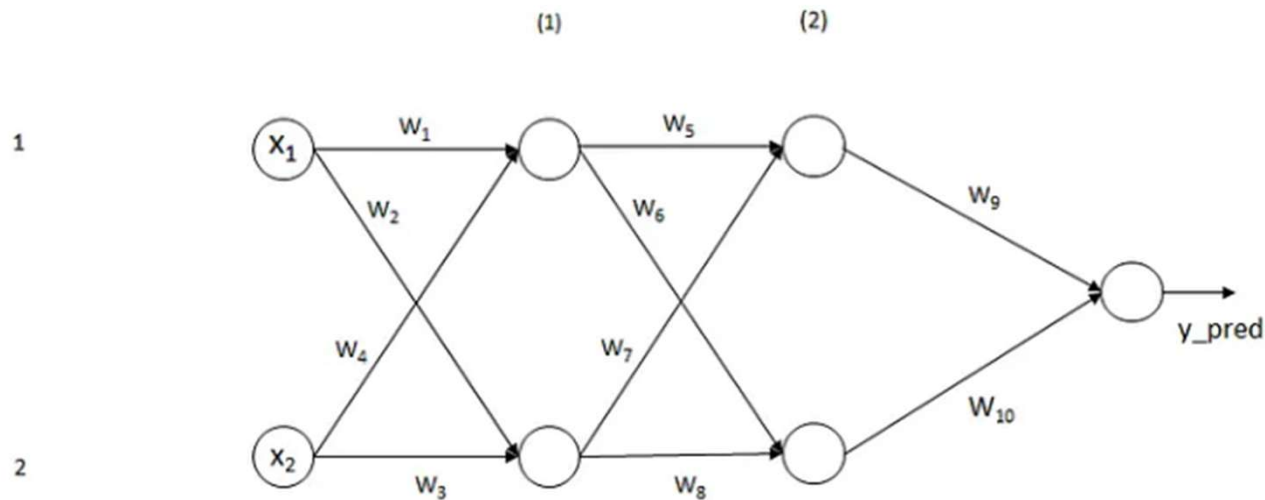
$$\Delta w = \eta(-\nabla_w L)$$

$$w_{new} = w_{old} - \eta \nabla_w L$$

Numero di iterate = numero di epoche

Data la funzione L differenziabile in $w_1 \dots w_m, b_1, \dots, b_n$, il suo gradiente esprime la direzione di crescita. Antepoendo il segno negativo siamo in grado di ottenere la direzione di discesa della stessa.

Ottimizzazione della discesa del gradiente

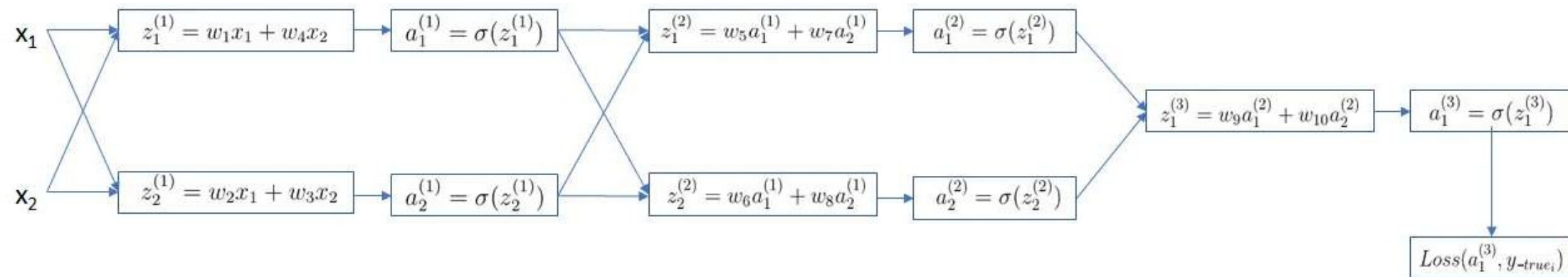
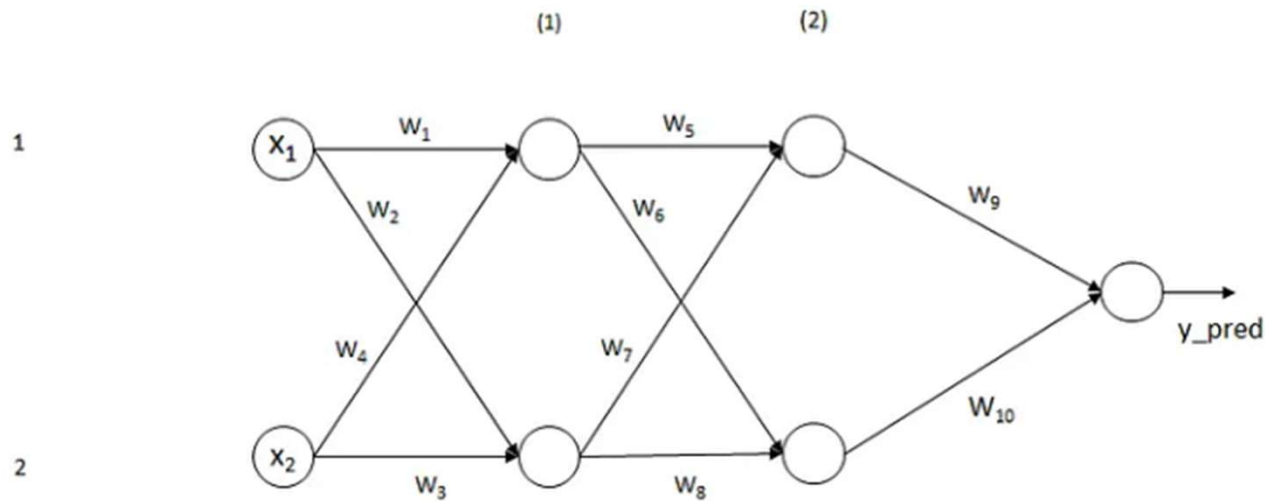


$$f(x, W) = y_{pred} = \sigma(w_9 a_1^{(2)} + w_{10} a_2^{(2)}) =$$

$$= \sigma(w_9 \sigma(w_5 a_1^{(1)} + w_7 a_2^{(1)}) + w_{10} \sigma(w_6 a_1^{(1)} + w_8 a_2^{(1)})) =$$

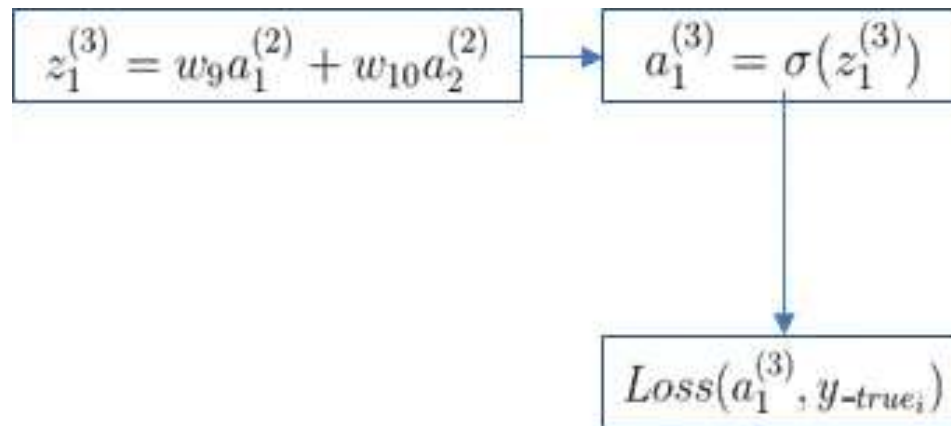
$$= \sigma(w_9 \sigma(w_5 \sigma(w_1 x_1 + w_4 x_2) + w_7 \sigma(w_2 x_1 + w_3 x_2)) + w_{10} \sigma(w_6 \sigma(w_1 x_1 + w_4 x_2) + w_8 \sigma(w_2 x_1 + w_3 x_2)))$$

Ottimizzazione della discesa del gradiente



Ottimizzazione della discesa del gradiente

Proviamo a calcolare la derivata della Loss rispetto w_9 . Le parti interessate del computational graph sono:

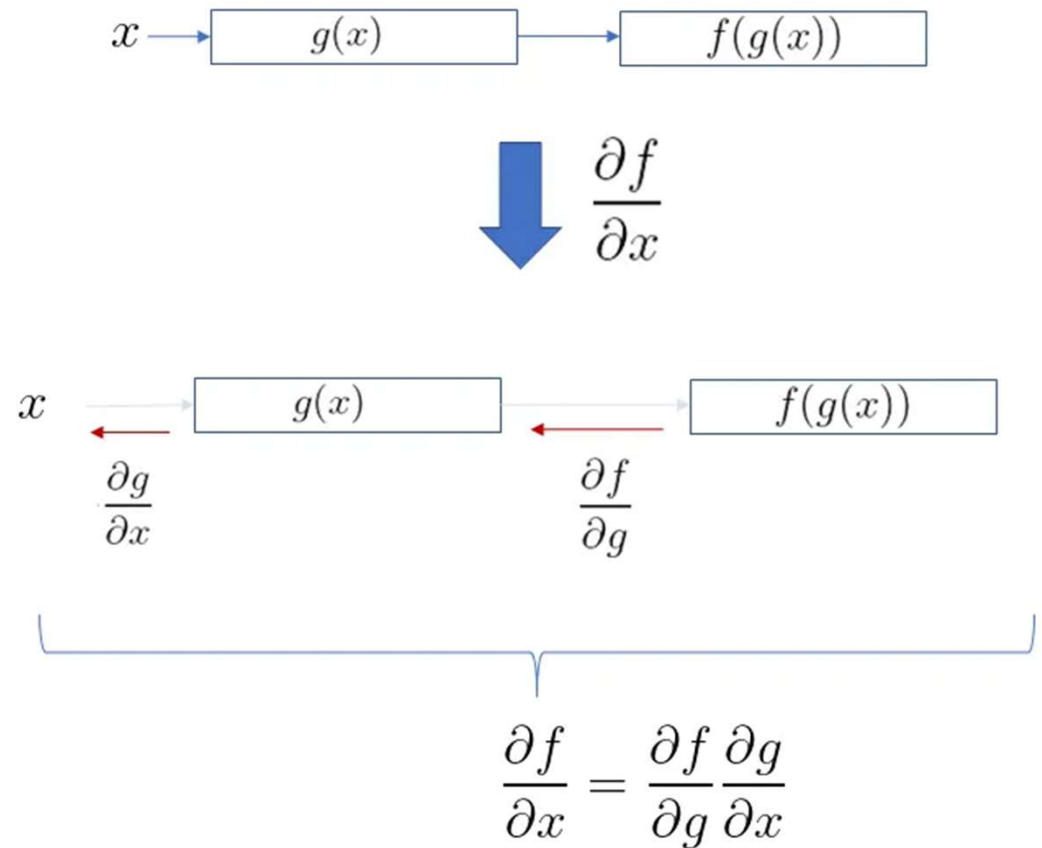


$$L(\sigma(w_9 a_1^{(2)} + w_{10} a_2^{(2)}))$$

Ottimizzazione della discesa del gradiente

Regola della catena (chain rule)

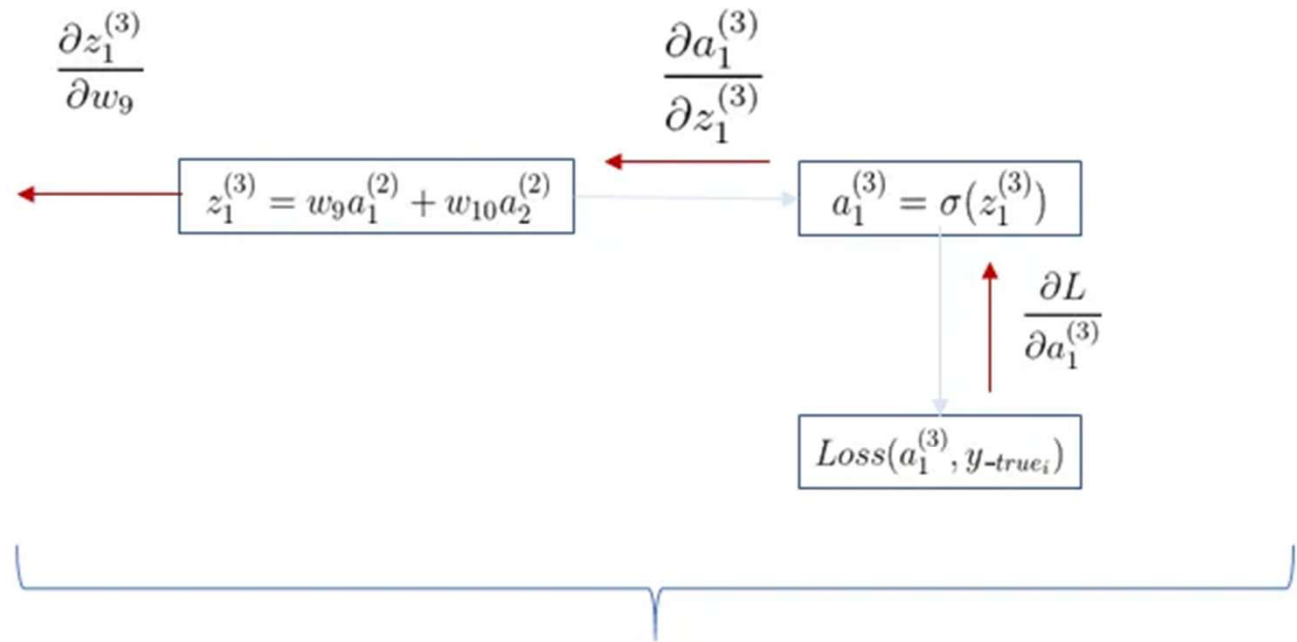
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$$



Ottimizzazione della discesa del gradiente

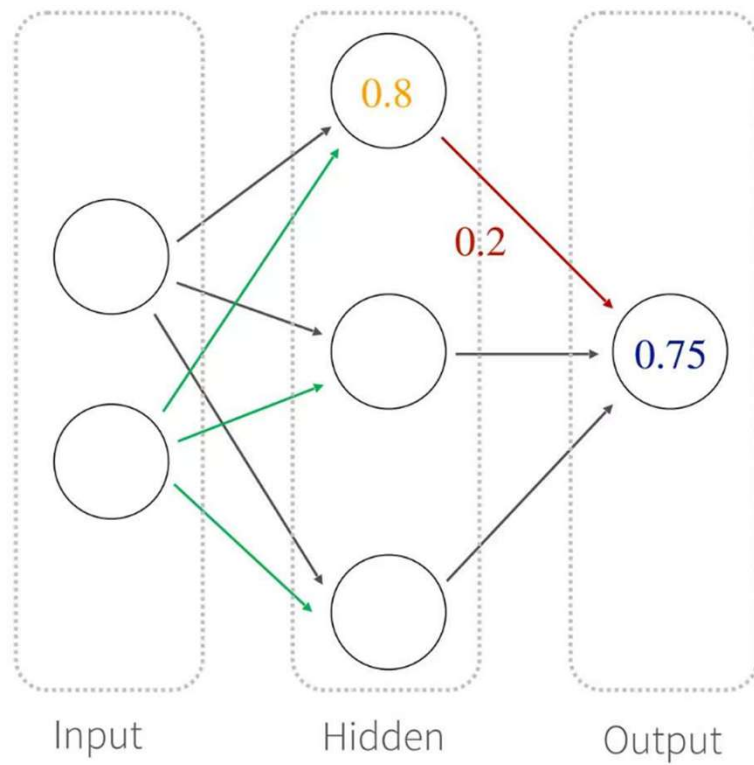
Regola della catena (chain rule)

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$$

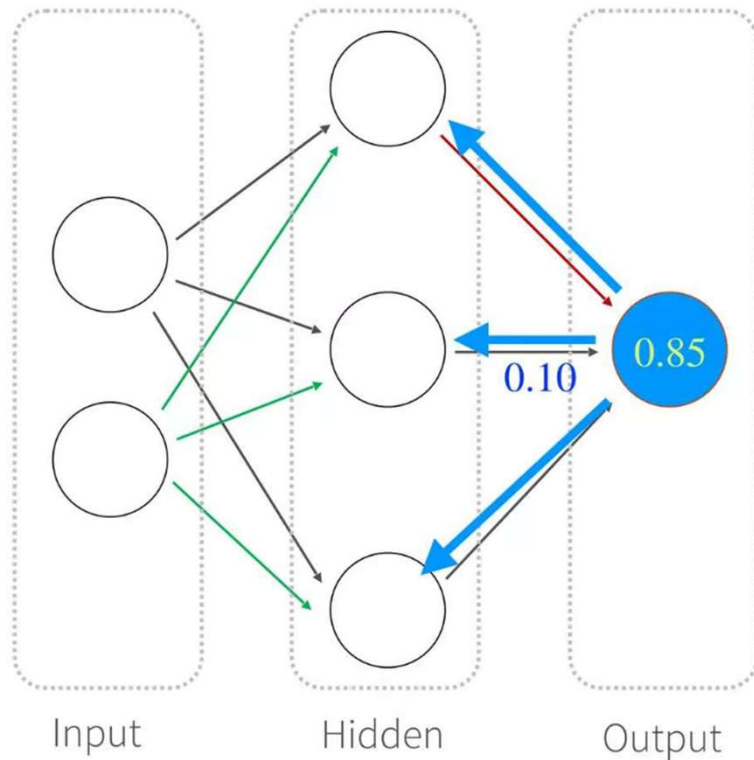


$$\frac{\partial L}{\partial w_9} = \frac{\partial L}{\partial a_1^{(3)}} \frac{\partial a_1^{(3)}}{\partial z_1^{(3)}} \frac{\partial z_1^{(3)}}{\partial w_9}$$

Retropropagazione (*Backpropagation*)



Retropropagazione (*Backpropagation*)

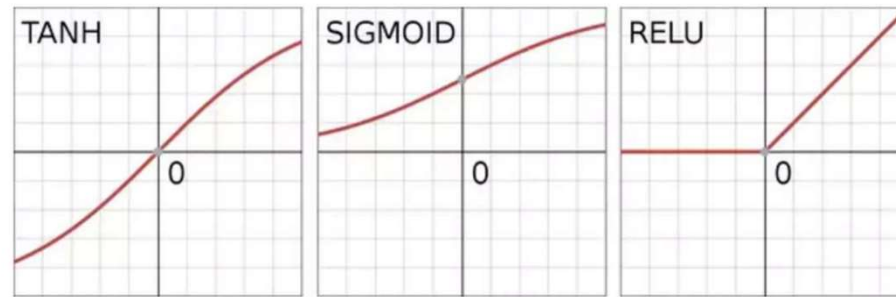
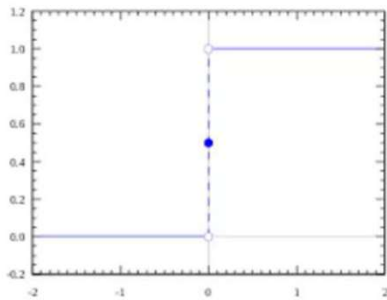


- Backpropagation: calcolare il gradiente della funzione di costo in una rete neurale
- Usato da algoritmi di ottimizzazione della discesa del gradiente per aggiornare i pesi dei neuroni
- Conosciuto anche come propagazione all'indietro degli errori dato che gli errori sono calcolati e distribuiti indietro attraverso i layer della rete

Neurons ... Activate!

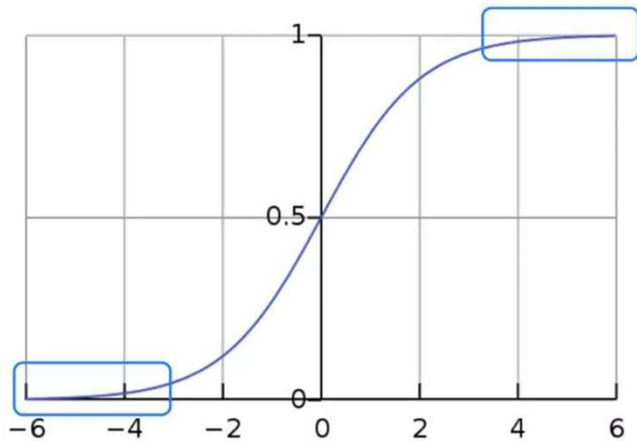


Funzioni di attivazione



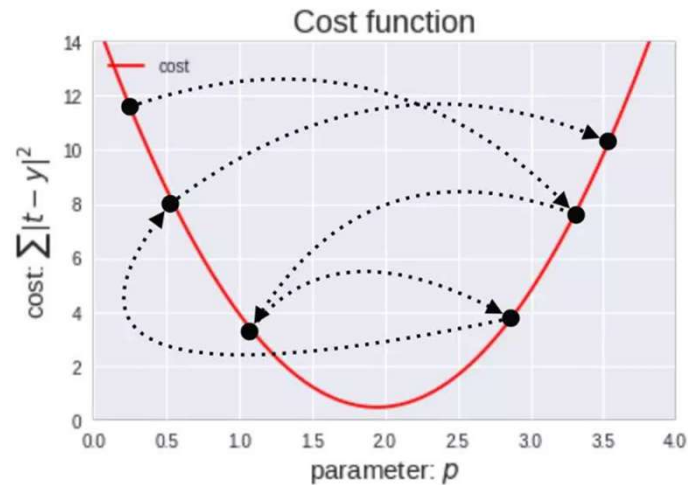
- Bias (soglia) prima funzione di attivazione proposta
- Sigmoide e tangente iperbolica (*tanh*) inducono non-linearità con codomini diversi
- ReLU è una delle più popolari perchè è semplice da calcolare e molto robusta al rumore nell'input

Funzione Sigmoidale



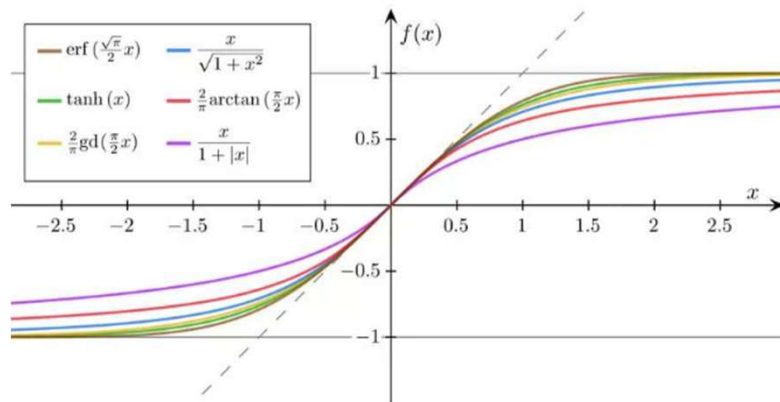
- La non-linearità della Sigmoidale appiattisce i numeri reali tra $[0, 1]$
- Storicamente una buona interpretazione dell'attivazione dei neuroni (cioè da non attivarsi affatto a un'attivazione satura).
- Attualmente, non molto usati perchè valori molto grandi in positivo o in negativo causano il problema del gradiente evanescente (*vanishing gradient*)

Funzione Sigmoida



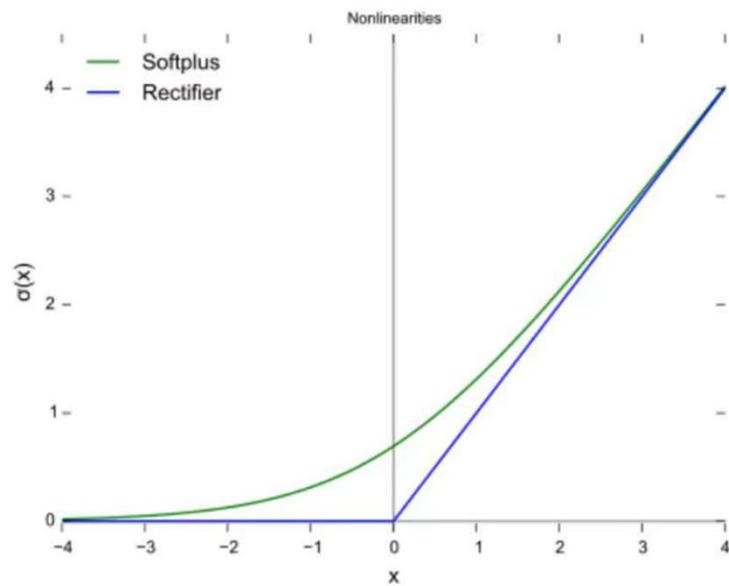
L'output non è centrato sullo zero: durante la discesa del gradiente, se tutti i valori sono positivi durante la backpropagation, i pesi diventeranno tutti positivi o tutti negativi creando una dinamica a zig zag

Funzione Tanh



- La funzione Tanh appiattisce i numeri reali nell'intervallo $[-1, 1]$
- Lo stesso problema della sigmoide: la sua saturazione può uccidere i gradienti
- Ma è centrata sullo zero, minimizzando la dinamica del zig zag durante la discesa del gradiente
- Attualmente è la funzione nonlineare preferita tra quelle simili alla sigmoide

Funzione rettificatore (ReLU)



L'attivazione di ReLU è alla soglia dello zero

- Parecchio popolare durante gli ultimi anni
- Velocizza la convergenza della discesa del gradiente stocastica (SGD)
- Semplice da implementare grazie alla sua semplice funzione matematica
- Sensibile a learning rate alti durante il training, causando neuroni “morti” (cioè neuroni che non si attiveranno mai per tutto il dataset).